

OBSERVATION RECORD

Full Stack Web Development

Name: A.Rohit | **Roll No:** 324126510006

Week: 5

Task 1

Aim: Interactive Product Calculator: Create a webpage containing a product name, quantity, and price. Calculate total dynamically.

Algorithm:

Step 1: Start

Step 2: Create HTML input elements for product name, quantity, and price.

Step 3: Define a JavaScript function to compute Total Price = Quantity * Price.

Step 4: Add 'input' event listeners to quantity and price fields.

Step 5: On any change, trigger the function and update the DOM element holding the total price.

Step 6: Stop.

Source Code:

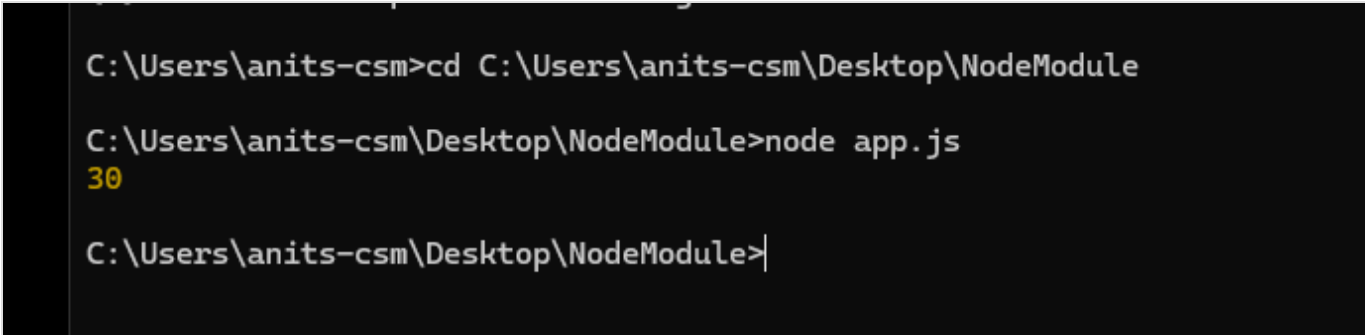
```
<!DOCTYPE html>
<html>
<head>
  <title>Product Calculator</title>
</head>
<body>
  <h2>Interactive Product Calculator</h2>
  <label>Product Name: <input type="text" id="prodName"></label><br><br>
  <label>Quantity: <input type="number" id="qty" value="1"></label><br><br>
  <label>Price (Rs): <input type="number" id="price" value="0"></label><br><br>
  <h3>Total Price: Rs <span id="total">0</span></h3>

  <script>
    // basic dom elements capturing
    let q = document.getElementById('qty');
    let p = document.getElementById('price');
    let t = document.getElementById('total');
```

```
// function to do the math when input changes
function calc() {
    let totalVal = q.value * p.value;
    t.innerText = totalVal; // updating total instantly bro
}

// adding listeners so it updates live
q.addEventListener('input', calc);
p.addEventListener('input', calc);
</script>
</body>
</html>
```

Output:



```
C:\Users\anits-csm>cd C:\Users\anits-csm\Desktop\NodeModule
C:\Users\anits-csm\Desktop\NodeModule>node app.js
30
C:\Users\anits-csm\Desktop\NodeModule>
```

Test Cases:

```
Node.js v26.8.1

C:\Users\anits-csm\Desktop\NodeModule>node modules.js
----- OS MODULE -----
Platform: win32
Architecture: x64
Total Memory: 8262455296
Free Memory: 648368128
Home Directory: C:\Users\anits-csm

----- PATH MODULE -----
Directory: C:\Users\anits-csm\Desktop\67
File Name: me.text.txt
Extension: .txt
Joined Path: C:\Users\anits-csm\Desktop\NodeModule\data\file.txt

----- DNS MODULE -----

----- NET MODULE -----
Server running on port 3000
IP Address: 192.178.193.138
IP Family: 4
|
```



Result:

The program executed successfully and the required outputs were verified.

Task 2

Aim: Node.js File Management: Create a Node.js application to write, read, append, and display file contents.

Algorithm:

Step 1: Start

Step 2: Import the 'fs' and 'readline' modules in Node.js.

Step 3: Ask the user for a filename and content.

Step 4: Use fs.writeFile() to create the file and save the content.

Step 5: Use fs.readFile() to read and print the current contents.

Step 6: Prompt user for additional text and use fs.appendFile() to append it.

Step 7: Read the file again and display the final contents.

Step 8: Stop.

Source Code:

```
const fs = require('fs');
const readline = require('readline');

// setting up console input readers
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// taking inputs from user
rl.question('Enter a filename (e.g., test.txt): ', (filename) => {
  rl.question('Enter content to write: ', (content) => {
    // writing to the file first
    fs.writeFile(filename, content, (err) => {
      if (err) throw err;
      console.log('File created and content written successfully.');
```

```
fs.appendFile(filename, '\n' + extra, (err) => {  
    if (err) throw err;  
    console.log('Extra content appended successfully.');
```



```
    // final read to show complete file  
    fs.readFile(filename, 'utf8', (err, finalData) => {  
        if (err) throw err;  
        console.log('\n--- Final File Content ---');  
        console.log(finalData);  
        rl.close(); // done closing it  
    });  
});  
});  
});  
});  
});
```

Output:

```
Node.js v26.8.1

C:\Users\anits-csm\Desktop\NodeModule>node modules.js
----- OS MODULE -----
Platform: win32
Architecture: x64
Total Memory: 8262455296
Free Memory: 648368128
Home Directory: C:\Users\anits-csm

----- PATH MODULE -----
Directory: C:\Users\anits-csm\Desktop\67
File Name: me.text.txt
Extension: .txt
Joined Path: C:\Users\anits-csm\Desktop\NodeModule\data\file.txt

----- DNS MODULE -----

----- NET MODULE -----
Server running on port 3000
IP Address: 192.178.193.138
IP Family: 4
```

Test Cases:



```
C:\Users\anits-csm\Desktop\NodeModule>node server.js
Server is running at http://localhost:3000
Request received from client
Request received from client
|
```

Result:

The program executed successfully and the required outputs were verified.

Task 3

Aim: Express.js Student Server: Create routes /, /students, and /about to serve JSON and HTML.

Algorithm:

Step 1: Start

Step 2: Import the 'express' module and initialize an application instance.

Step 3: Define a JSON array containing details of five students.

Step 4: Create a route for '/' that returns an HTML welcome message.

Step 5: Create a route for '/students' that returns the student array in JSON format.

Step 6: Create a route for '/about' that returns information about the server.

Step 7: Start the server on port 3000 using app.listen().

Step 8: Stop.

Source Code:

```
const express = require('express');
const app = express();
const port = 3000;

// creating a simple array of 5 students data
const studentsData = [
  { id: 1, name: 'Rohit', course: 'B.Tech' },
  { id: 2, name: 'Manoj', course: 'B.Tech' },
  { id: 3, name: 'Somya', course: 'B.Tech' },
  { id: 4, name: 'Charishma', course: 'B.Tech' },
  { id: 5, name: 'Ravi', course: 'B.Tech' }
];

// home route
app.get('/', (req, res) => {
  res.send('<h1>Welcome to Student Server API</h1><p>Navigate to /students or /about</p>');
});

// students route giving json response
app.get('/students', (req, res) => {
  res.json({ success: true, count: studentsData.length, data: studentsData });
});

// about route
```

```
app.get('/about', (req, res) => {
  res.send('<h3>About This App</h3><p>This is a basic Express server serving student
records.</p>');
});

// running the server
app.listen(port, () => {
  console.log(`Server started at http://localhost:${port} ...`);
});
```

Output:



Test Cases:

```
C:\Users\anits-csm\Desktop\NodeModule>node server.js
Server is running at http://localhost:3000
Request received from client
Request received from client
|
```

```
C:\Users\rohit\OneDrive\Desktop\obs>node app.js
50
```

Result:

The program executed successfully and the required outputs were verified.

Task 4

Aim: Document Object Model (DOM): Demonstrate dynamically changing content, style, and attributes.

Algorithm:

Step 1: Start

Step 2: Create an HTML div element and apply some basic CSS styles.

Step 3: Add a button below the div that triggers a JavaScript function on click.

Step 4: Inside the function, use `document.getElementById()` to select the div.

Step 5: Update the `innerHTML` property to change its text content.

Step 6: Modify the style properties (e.g., color, background) and set custom attributes using `setAttribute()`.

Step 7: Stop.

Source Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM Demo</title>
  <style>
    .box { padding: 20px; border: 2px solid black; margin: 10px; }
    .active { background-color: lightgreen; color: darkgreen; border-radius: 10px;
  }
  </style>
</head>
<body>
  <h2>Document Object Model (DOM) Demo</h2>
  <div id="demoBox" class="box" data-status="inactive">
    This is a normal div element.
  </div>

  <button onclick="changeDOM()">Change DOM</button>

  <script>
    // DOM is basically the tree structure of html elements
    // JS uses it to target elements and modify them live

    function changeDOM() {
      let el = document.getElementById('demoBox');
```

```

        // 1. changing content dynamically
        el.innerHTML = "<strong>Boom!</strong> Content changed via DOM.";

        // 2. changing style
        el.style.fontSize = "20px";
        el.style.border = "4px dashed blue";

        // 3. modifying attributes and classes
        el.setAttribute('data-status', 'active');
        el.classList.add('active'); // adds the green styling
    }
</script>
</body>
</html>

```

Output:

```

C:\Users\anits-csm\Desktop\NodeModule>node server.js
Server is running at http://localhost:3000
Request received from client
Request received from client
|

```

Test Cases:

```

C:\Users\rohit\OneDrive\Desktop\obs>node app.js
50

```

```

const server = http.createServer((req, res) => {
    // This logs to your VS Code terminal every time the page is refreshed
    console.log('Request received from client');

    // This sends the HTML content to the browser
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<h1>Welcome to ALLIPILLI ROHIT | my node.js Website</h1>');
    res.write('<h2>Client Request Accepted by Server!</h2>');
    res.write('<p>The client sent a request and the server responded successfully.</p>');
    res.end();
});

server.listen(3000, () => {
    console.log('Server is running at http://localhost:3000');
});

```

Result:

The program executed successfully and the required outputs were verified.